

TCP

Recap: *TCP within the overall context of
the TCP/IP internetworking model*

כפי שראינו בהרחבה רבה בהרצאות האחרונות, מודל ה- TCP/IP תוכנן ומומש מתוך מטרה לאפשר חיבור של רשתות שונות לרשת וירטואלית אחת גדולה
רשתות שונות = רשתות ארגוניות שונות
רשת וירטואלית = אין 'באמת' רשת אחת גדולה, אלא יש הרבה מאוד {עשרות/מאות מיליונים} של רשתות שונות.

במילים אחרות: המודל 'יוצר'/מממש אשליה {fake} של רשת אחת גדולה
עבור מי האשליה הזו מיועדת ? עבור אפליקציות רשתיות
שהן מספקות שירותי העברת מידע עבור משתמשי הקצה

- בין יתר 'המרכיבים' של המודל ראינו כי נכללים:

1. שירות של העברת מידע *host-to-host*: העברת מידע בין כל שני מארחים שונים ש'ששיכים' לרשת הווירטואלית.

- בין אם הם שייכים לאותה רשת ארגונית, ובים אם לרשתות ארגוניות שונות.

2. שירות של העברת מידע *end-to-end*: העברת מידע בין כל שתי תוכניות שונות שרצות במארחים שונים ש'ששיכים' לרשת הווירטואלית.

שכבת ה- *network* [= שכבה מספר 3], היא האחראית 'לספק' את ה- *host-to-host service* בשכבה זו, פרוטוקול ה- IP הוא הפרוטוקול העיקרי. עיקרי אבל לא יחיד. יש פרוטוקולים נוספים. כמו לדוגמא פרוטוקול ה- NAT.

שכבת ה- *transport* [= שכבה מספר 4], היא האחראית 'לספק' את ה- *end-to-end service*. שכבה זו כוללת לצורך כך 2 פרוטוקולים: UDP ו- TCP

שכבת ה-transport, TCP ו-UDP, מספקת שירות לשכבת האפליקציה [= שכבה מספר 5]
או, ליתר דיוק: לפרוטוקולים של שכבת האפליקציה

- 2 סוגים של שירותים:

- שירות שהוא גם connection oriented וגם העברת נתונים אמינה [=RDT]. TCP.
- שירות שהוא גם connectionless וגם העברת נתונים לא אמינה. UDP.

- השירות ששכבה ה-Transport מספקת לפרוטוקולי שכבה 5 'מונגש' דרך/ע"י ממשק הסוקטים [= sockets API]

- סוגים שונים של sockets APIs תוכננו ומומשו. ובכלל זה:

- BSD sockets

- Winsock

בקורס אנחנו מתמקדים ב-BSD sockets API.

פרוטוקולי שכבה 5 אינם 'מתקיימים' ב'פני עצמם', אלא הם חלק מאפליקציות רשתיות

אפליקציה רשתית = לפחות 2 תכונות שונות שרצות במארחים שונים ששייכים לרשת הווירטואלית התכניות השונות מחליפות מידע לפי אוסף כללים/חוקים שנקרא פרוטוקול תקשורת. פרוטוקול שכבה 5. קיימים מודלים שונים של אפליקציות רשתיות מודל לקוח-שרת מודל עמית-לעמית מודל משולב אנחנו בקורס מתמקדים באפליקציות לקוח-שרת בלבד.

באפליקציות אלו תכנית אחת נקראת תכנית הלקוח והשנייה נקראת תכנית השרת. הן, כאמור, מחליפות הודעות ביניהן לפי הכללים של פרוטוקול שכבה 5. שגם הם, מטבע הדברים, מייצג מודל לקוח-שרת: לפרוטוקול יש צד לקוח ויש צד שרת יחידות המידע המוחלפות בין צד הלקוח של הפרוטוקול ובין צד השרת שלו נקראות הודעות.

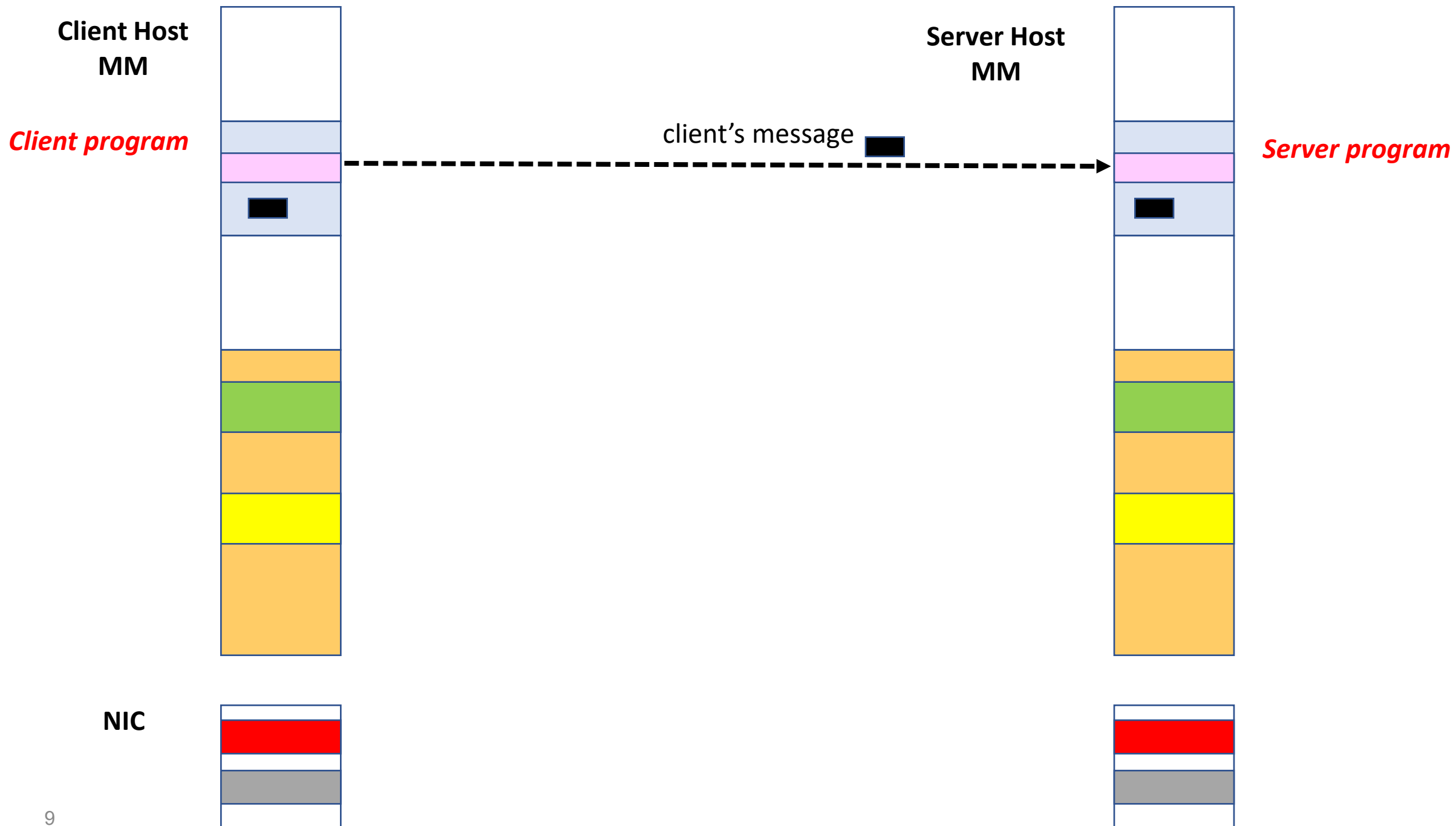
- ואם 'מחברים' את כל שנאמר כד כה:
- שכבת ה-Transport מספקת שירות של end-to-end לפרוטוקולי שכבה 5.
- ז"א, שכבת ה-transport לא מספקת שירות של end-to-end ישירות לאפליקציות הרשתיות. אלא,
'בעקיפין': 'דרך' פרוטוקולי שכבה 5 שמממשים את העברת ההודעות בין 'חלקי האפליקציה' השונים [=]
בין תכנית הלקוח ותכנת השרת].

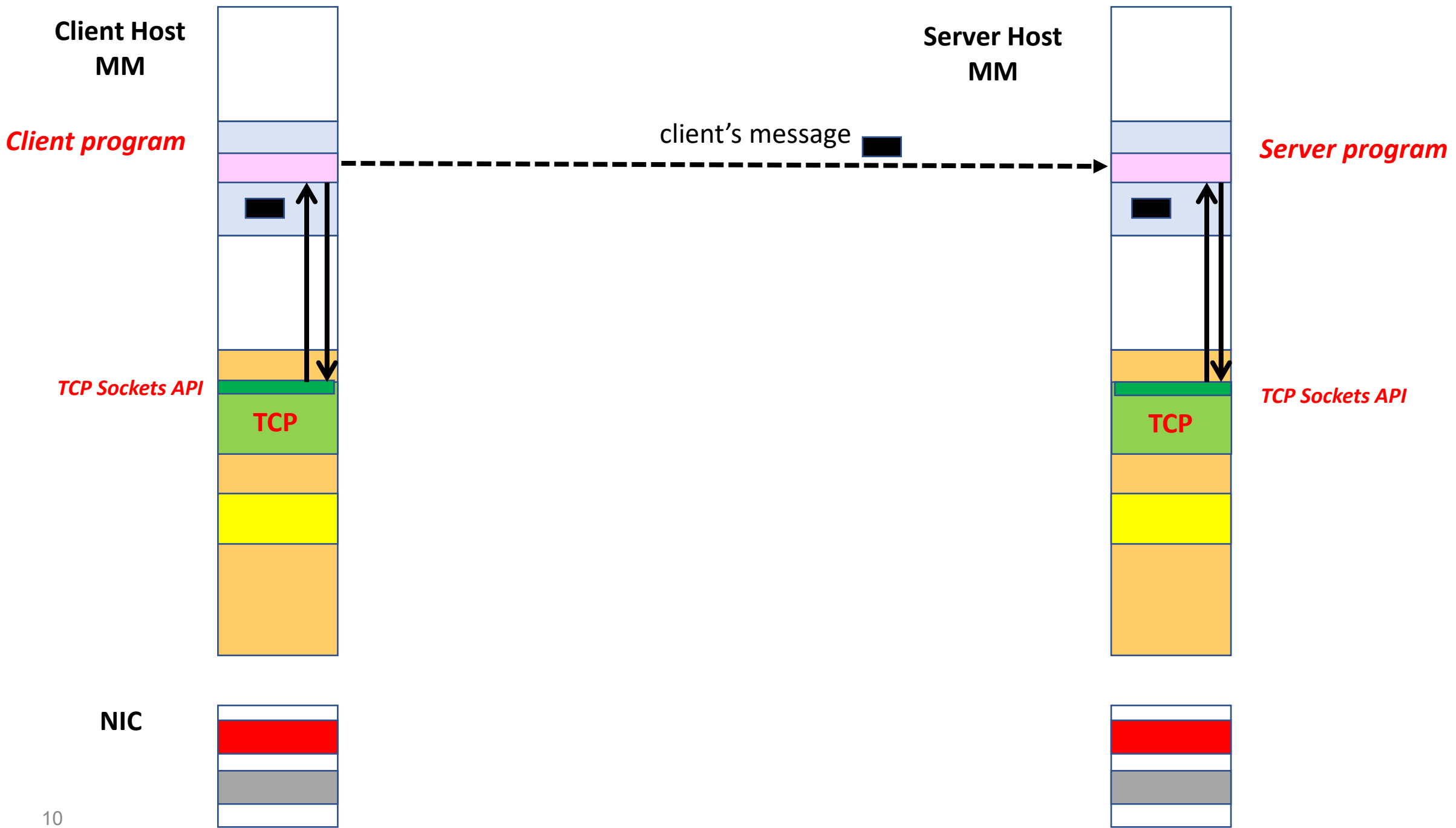
- והשירות 'מונגש' לפרוטוקולי שכבה 5 על ידי ה- sockets API
- ולפיכך, יש 2 סוגי סוקטים: סוקטים של TCP וסוקטים של UDP
- ולכן, יש פונקציה [socket() =] שמיועדת ליצירת/הקצאת סוקט. שמאפשרת יצירת סוקט TCP או
UDP.

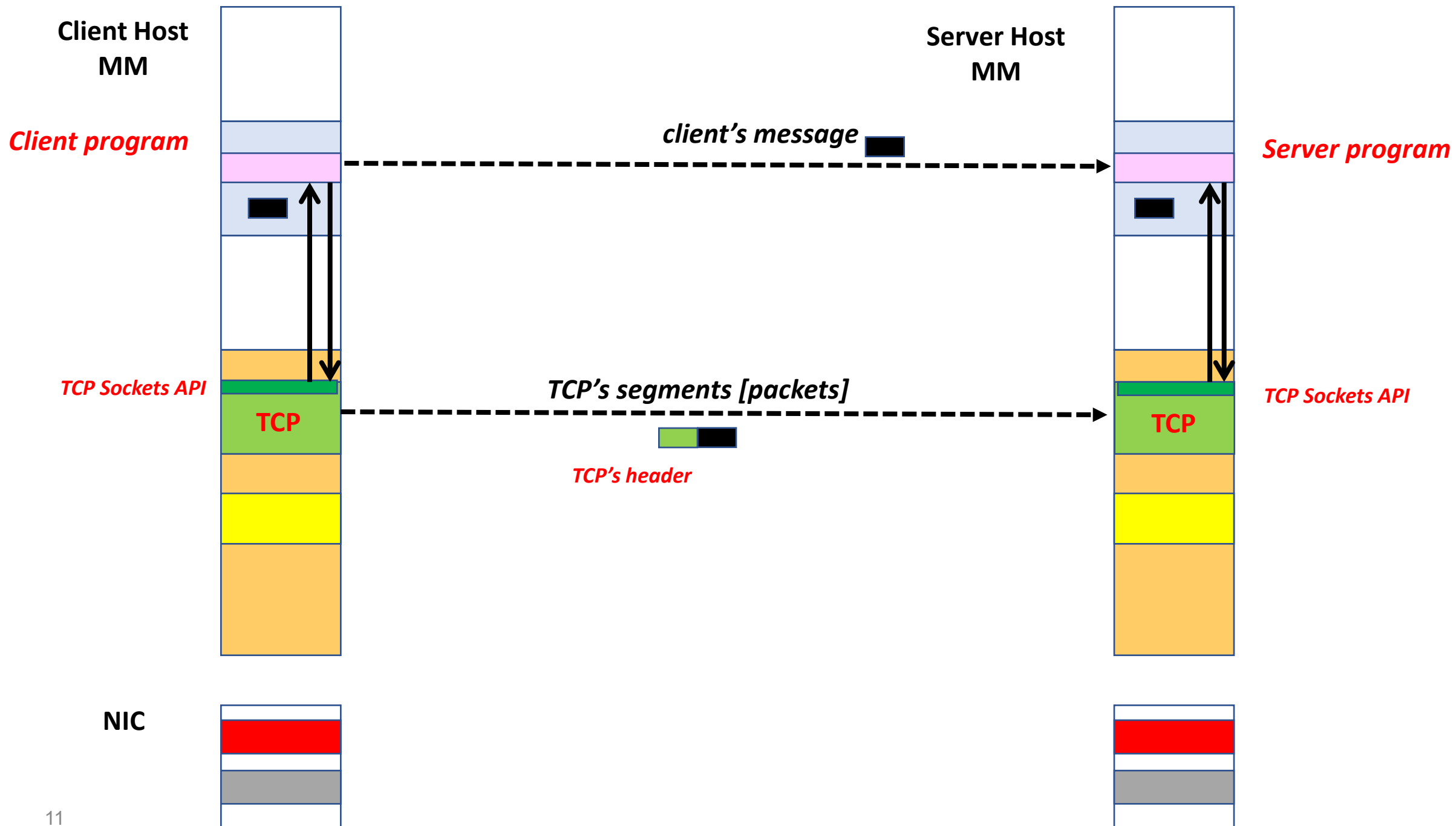
- ויש פעולות [= פונקציות] שמיועדות לסוקטים של TCP.
- לדוגמא: connect(), bind(), send(), receive(),
- ויש פעולות [= פונקציות] שמיועדות לסוקטים של UDP.
- sendto(), recvfrom(), ...

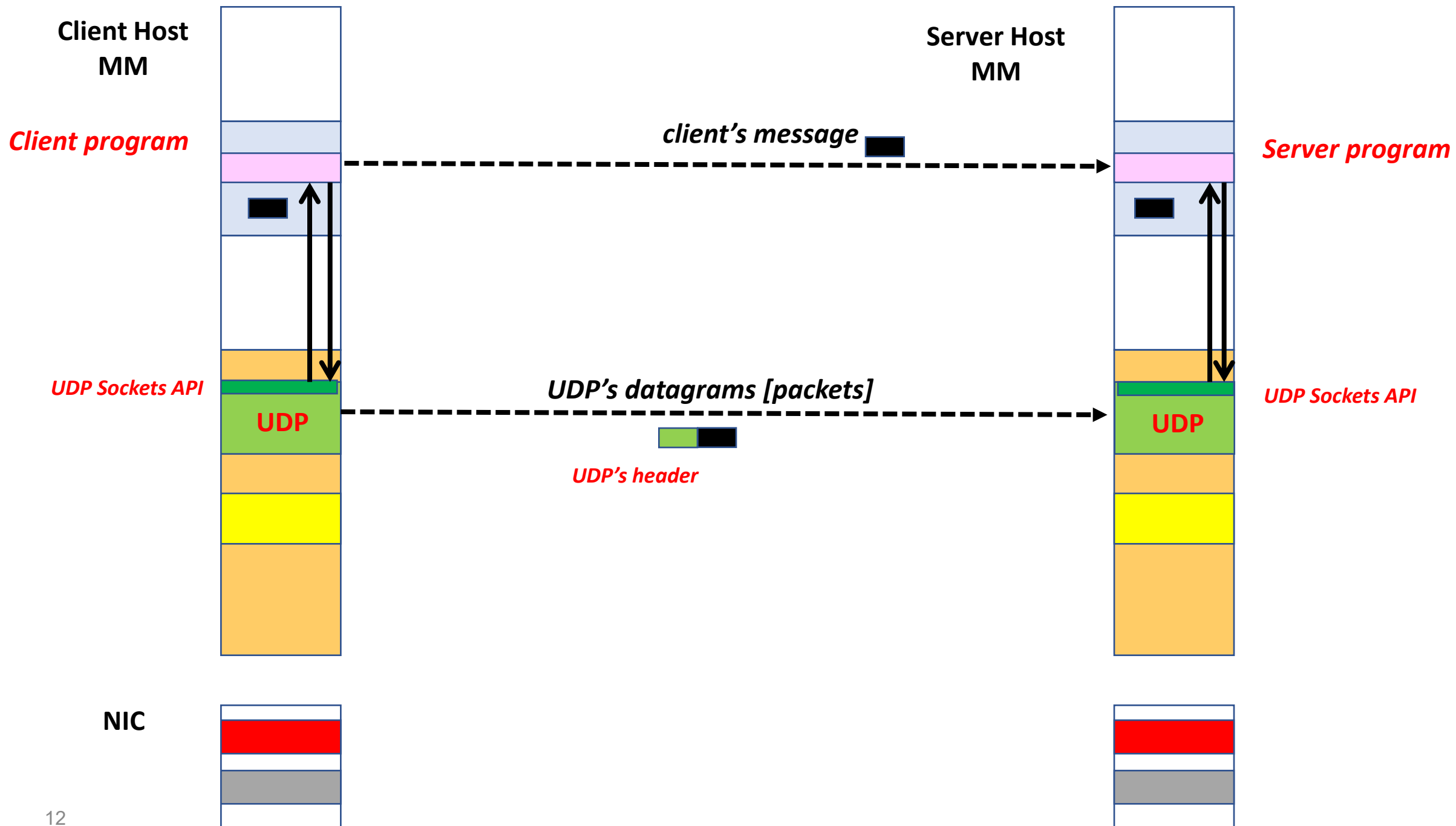
ומכלול הפונקציות השונות מהווה את ממשק ה- sockets API

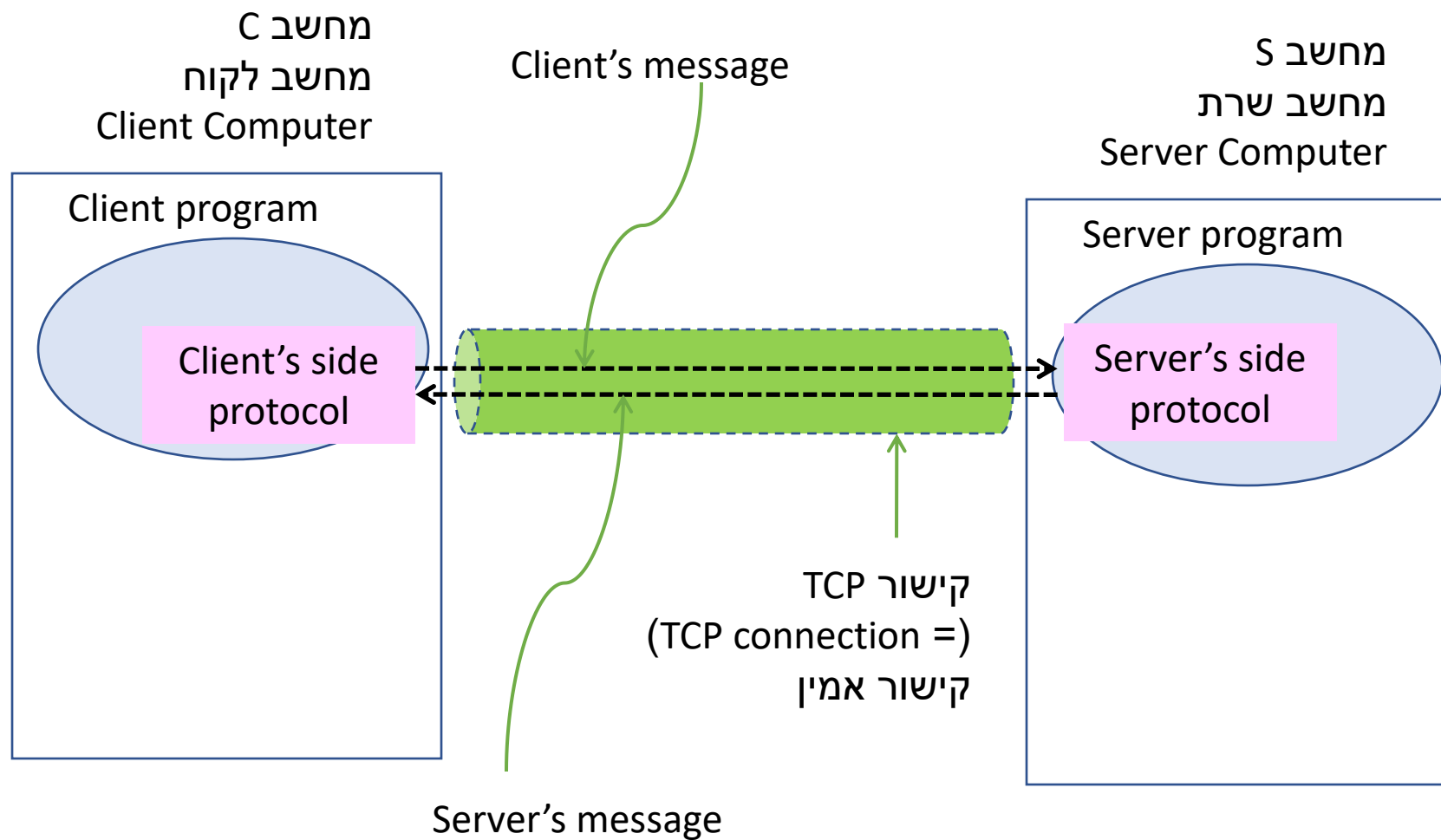
TCP {Virtual} Connection











‘Our Way’ to Describe TCP

TCP achieves a seemingly impossible task

- *TCP achieves a seemingly impossible task:*
 - *it uses the unreliable datagram service offered by IP when sending across the virtual network*
 - *e.g., Internet*
 - *but provides a reliable data delivery service to application programs.*

TCP's challenges

- *TCP must compensate for:*
 - *loss,*
 - *delay,*
 - *duplication,*
 - *and out-of-order delivery,*
- *and it must do so without overloading the underlying networks and routers.*
- *We first review the service that TCP provides to applications,*
- *Then we examine the techniques TCP uses to achieve reliability.*

לגבי אופן התיאור שלנו בקורס את פרוטוקול ה-TCP

- בהמשך לתזכורת שהובאה בשקפים הקודמים, הדיון שלנו בפרוטוקול ה-TCP 'ישתרע' על פני מספר צירים:

- ציר #1: תיאור כללי שיסקור את התכונות/המאפיינים השונים של הפרוטוקול ובכלל זה יצביע על האתגרים/הקושי לממש חלק מהתכונות האלו

- ציר #2: תיאור קונספטואלי שיתאר את האופן שבו TCP מממש את המאפיינים שלו. היינו, איך התכנון של TCP נותן מענה לאתגרים הכרוכים במימוש התכונות שלו

- ציר #3: ממשק הסוקטים ש-TCP 'מציע' לפרוטוקולי שכבה 5 שמחליפים ביניהם הודעות בעזרת/באמצעות TCP.

*TCP's Characteristics
[or: The Service TCP Provides to
Applications]*

TCP's Service

The service offered by TCP has seven major features:

1. Connection Orientation.

TCP provides connection-oriented service in which an application must first request a connection to a destination, and then use the connection to transfer data.

2. Point-To-Point Communication.

Each TCP connection has exactly two endpoints.

3. Complete Reliability.

TCP guarantees that the data sent across a connection will be delivered exactly as sent, complete and in order.

4. Full Duplex Communication.

A TCP connection allows data to flow in either direction, and allows either application program to send data at any time.

TCP's Service

5. Stream Interface.

TCP provides a stream interface, in which an application sends a continuous sequence of octets across a connection.

TCP does not group data into records or messages, and does not guarantee to deliver data in the same size pieces that were transferred by the sending application.

6. Reliable Connection Startup. *TCP allows two applications to reliably start communication.*

7. Graceful Connection Shutdown. *Before closing a connection, TCP ensures that all data has been delivered and that both sides have agreed to shut down the connection.*

Summary

- *TCP provides
a reliable,
connection-oriented,
full-duplex
stream
transport service that allows two application programs to form a connection, send data in either
direction, and then terminate the connection.*
- *Each TCP connection is started reliably and terminated gracefully.*

*The challenges/problems that the
designers of the TCP protocol had to
face/overcome*

An end-to-end transport protocol must be carefully designed to achieve efficient, reliable transfer.

The major problems are:

1. Unreliable Communication.

Elaboration: Data sent can be:

1.1 Received 'damaged' [= corrupted]

1.2 Get lost

1.3 duplicated

1.4 Received out of order

2. 'Old' Data arrival. The server program 'crashes', and then restarted.

3. Flow control

4. Congestion control

1.1 Received 'damaged' [= corrupted]

- *Data sent can be received 'damaged' [= corrupted]*
 - *That is: with errors.*
 - *That is: the data received by the receiver is 'different' from the data sent by the sender*
- *Reason: the carrier signal/s are 'damaged'.*
 - Due, for example, to environmental 'noises'*

1.2 Get lost

- *Data sent can be received 'damaged' [= corrupted]*
 - *That is: the data sent by the sender doesn't reach the sender.*
 - *At all !!*

Reason: one of the buffers in one of the communication devices along the route from the source [= sender] host to the destination hosts is full
Therefore, the communication device can not store the data until it forwards it toward the destination host

1.3 duplicated

- *Data sent can be duplicated.*
 - *That's is: 2 , or more, copies of the data are received*

Reason: retransmission due the {incorrect !!} suspect that it got loast, and therefore it must be retransmitted

1.4 received out-of-order

- *Data sent can be received out-of-order.*
 - *That's is: the data id received in a different order than the order it was sent*

Reason: A. arrival of 'damaged' data.

in which case it must be discarded by the receiver, and therefore must be retransmitted. But meanwhile {between the arrival of the damaged data, until the sender finds out that it was damaged and therefore retransmit it, and until the retransmitted data arrives} a 'latter sent' data arrives correctly.

B. Lost data

2. 'Old' Data arrival. The server program 'crashes', and then restarted.

When the server program crashes, the client programs, connected to the server program, don't get any notification.

Therefore, these programs 'thinks' that the server program is still running.

And continue to send data.

But from the server program's point of view, once it was crashed it doesn't 'remember' at all that it 'used to have' some 'old clients'

From the server program point of view the only clients it serves are those that were connected with it after the restart.

These 'new' clients are the ONLY clients.

A data sent by an old client may be delayed and received by the server program only after the server program had crashed, restarted and start serving 'new' clients.

This data may be mistakenly considered by the server program to be sent from one of the clients it currently serves. That is: from a 'new' client. While it was actually sent from an 'old' client.

The server program, does not distinguish between 'old' and 'new'. There are only the current clients it serves.

3. Flow control

Due to the fact that the sending host and the receiving host are heterogeneous, an application running on a powerful processor can generate data so fast that it overruns an application running on a slow processor.

d Congestion In The Internet. If senders aggressively transmit data, intermediate switches and routers can become overrun with packets, analogous to a congested highway.

4. Congestion control

--

*The techniques designed and adopted
by the TCP designers for the
challenges/problems that the protocol
exhibit*

In the previous slides we described the challenges/problems that the designers of the TCP protocol had to face/overcome

How does TCP provide solutions to above described challenges/problems?

What techniques it implements ?

TCP uses a variety of techniques {algorithms} and data structures that are combined in novel ways.

As expected TCP uses retransmission to compensate for packet loss.

Because TCP provides data flow in both directions, both sides of a communication participate in retransmission.

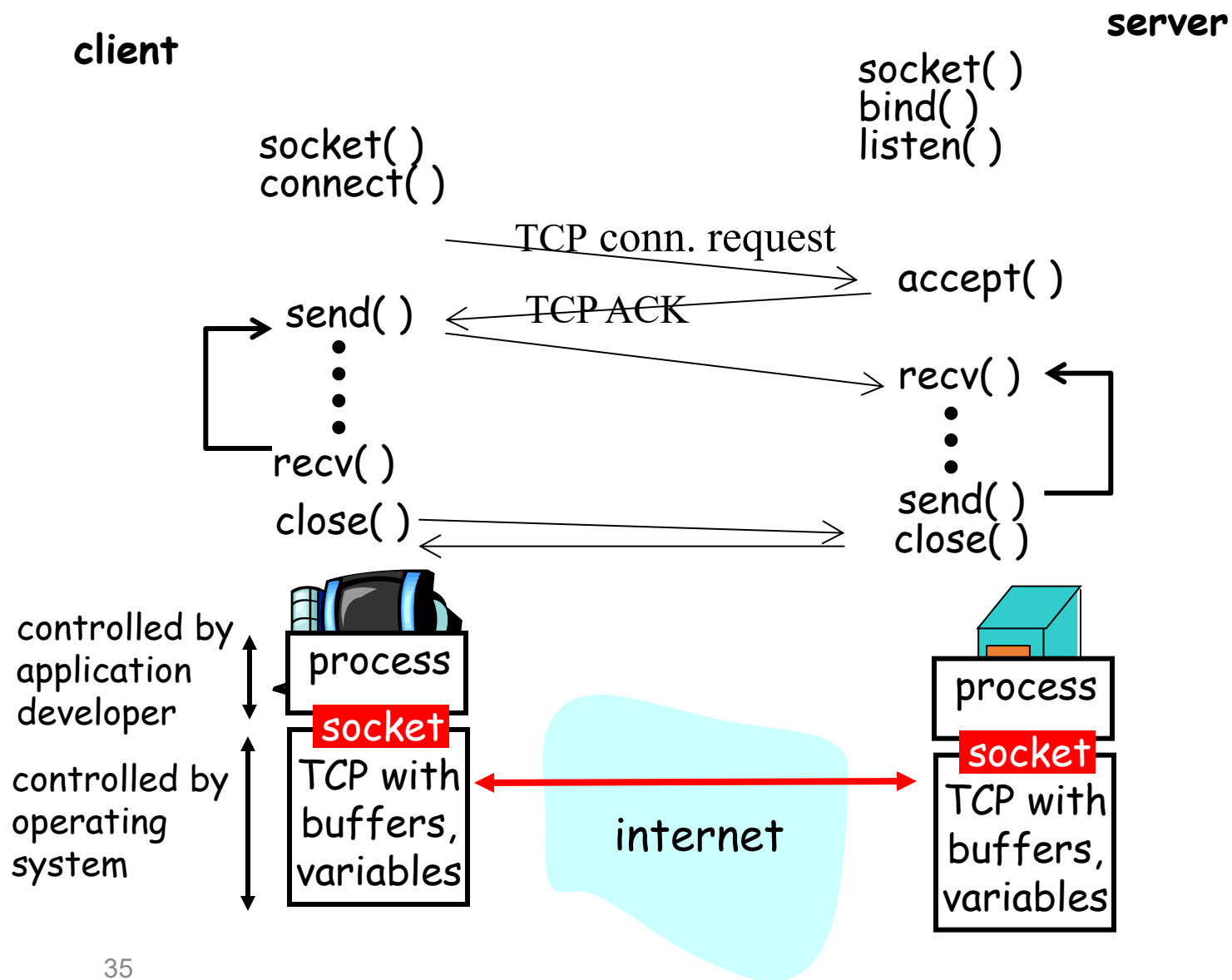
When TCP receives data, it sends an acknowledgement back to the sender.

Whenever it sends data, TCP starts a timer, and retransmits the data if the timer expires.

*An Overview Of
The TCP's Sockets API*

TCP client & server:
overall interaction

Socket-programming using TCP



- השקף מתאר את האינטראקציה (הכוללת !) בין תכנית לקוח, cli, לבית תכנית שרת, srv.
 - התכניות מחליפות ביניהן מידע [= הודעות] באמצעות קישור TCP
 היינו: באמצעות פרוטוקול תקשורת שכבה 5: P
 תיאור מדויק יותר מהנ"ל: צד הלקוח של הפרוטוקול, Pcli וצד השרת של הפרוטוקול, Psrv, מחליפים הודעות באמצעות קישור TCP

- האינטראקציה נעשית ע"י כך ש- Pcli ו- Psrv קוראות לפונקציות [= מבצעות קריאות מערכת] לדוגמא: socket(), bind(), ...
 - פונקציות אלו מהוות חלק מממשק הסוקטים שתואר מספר פעמים במהלך ההרצאות עד כה.
 - בין היתר, בהרצאה שעסקה בתיאור מחזור החיים של תכנון המודל הרשתי TCP/IP

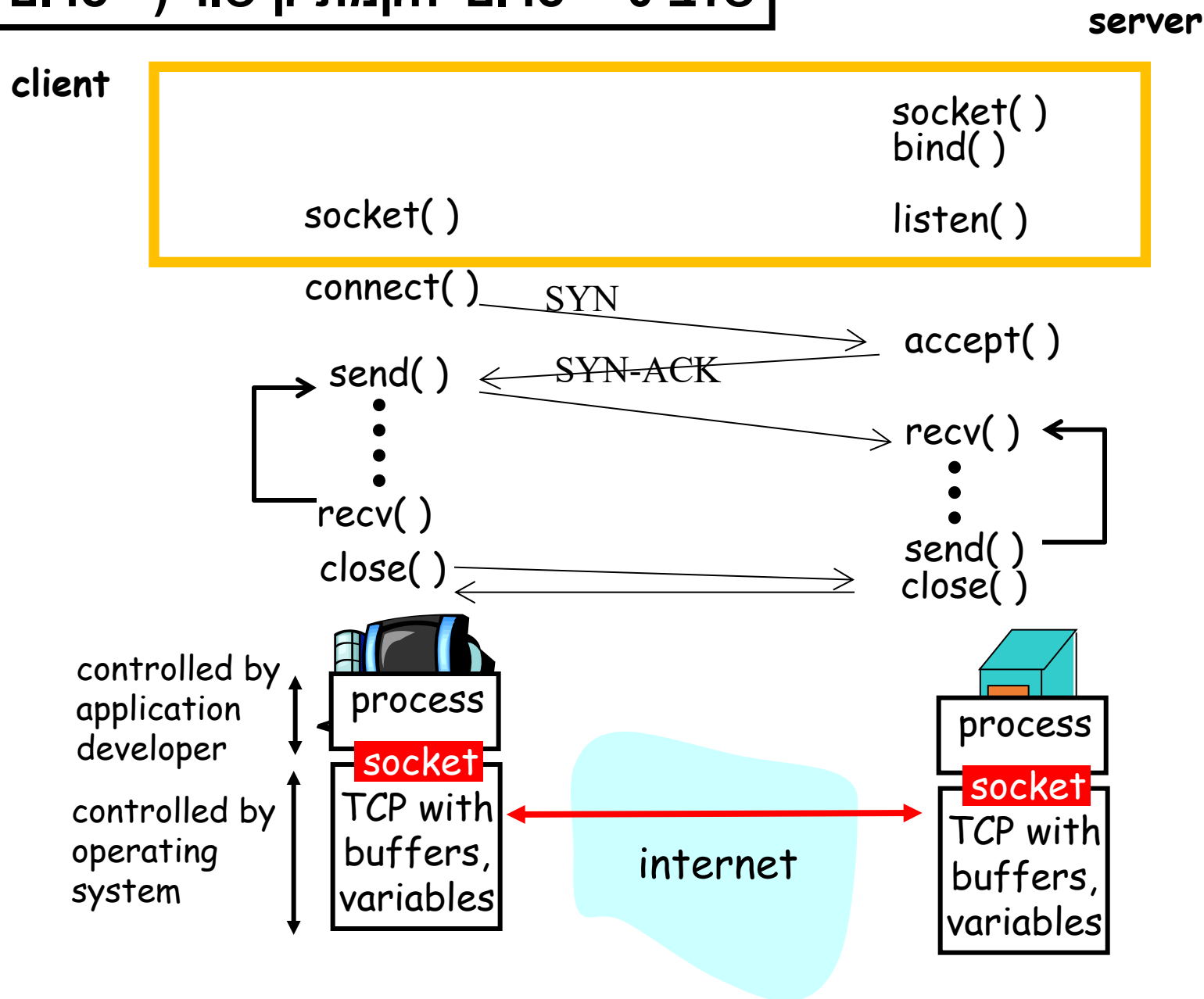
- המימוש של כל הפונקציות האלו (socket, bind, listen, ...) נעשה ע"י מודול ה- TCP שהוא, כזכור, חלק ממודול ה- TCP היינו, הפונקציות האלו הן חלק ממאות (רבות) של פונקציות ש'ביחד' מהוות את TCP [שהוא 'חלק' מעשרות ומאות אלפי פונקציות שביחד מהוות את מערכת ההפעלה]

האינטראקציה הכוללת בין לקוח TCP ושרת TCP

האינטראקציה הכוללת בין לקוח ושרת המחליפים ביניהם הודעות 'באמצעות' קישור TCP כוללת 4 שלבים:

- 0. 'טרום' לחיצת יד [= 'טרום' הקמת קישור]
- 1. הקמת קישור [= לחיצת יד תלת-שלבית]
- 2: החלפת דאטה בין הלקוח והשרת [= החלפת הודעות בקשה ותשובה]
- סגמנטים ש'מכילים' הודעות או 'חלקי' הודעות
- 3: סגירת הקישור.

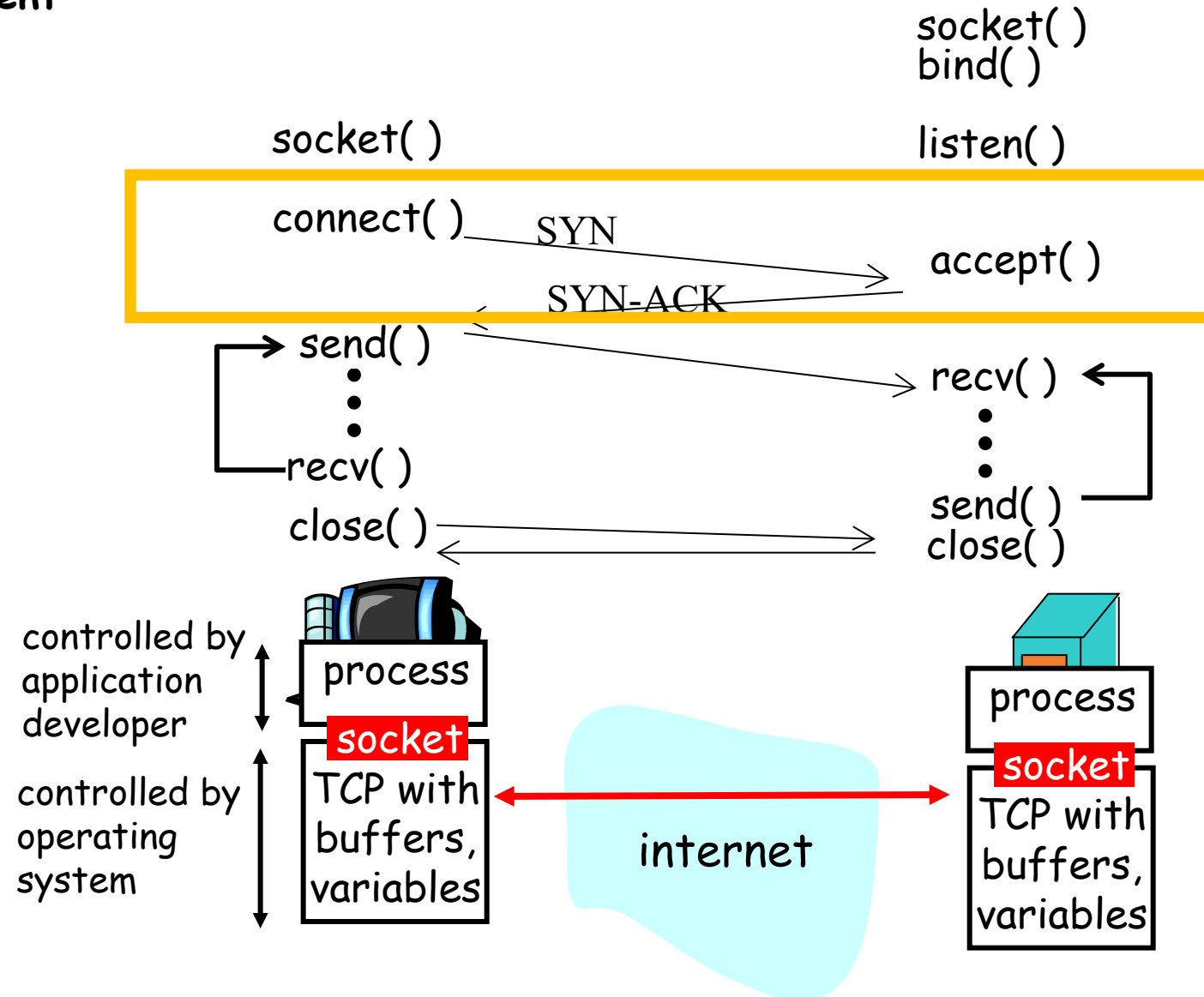
שלב 0 – 'טרום' הקמת קישור (= טרום לחיצת יד)



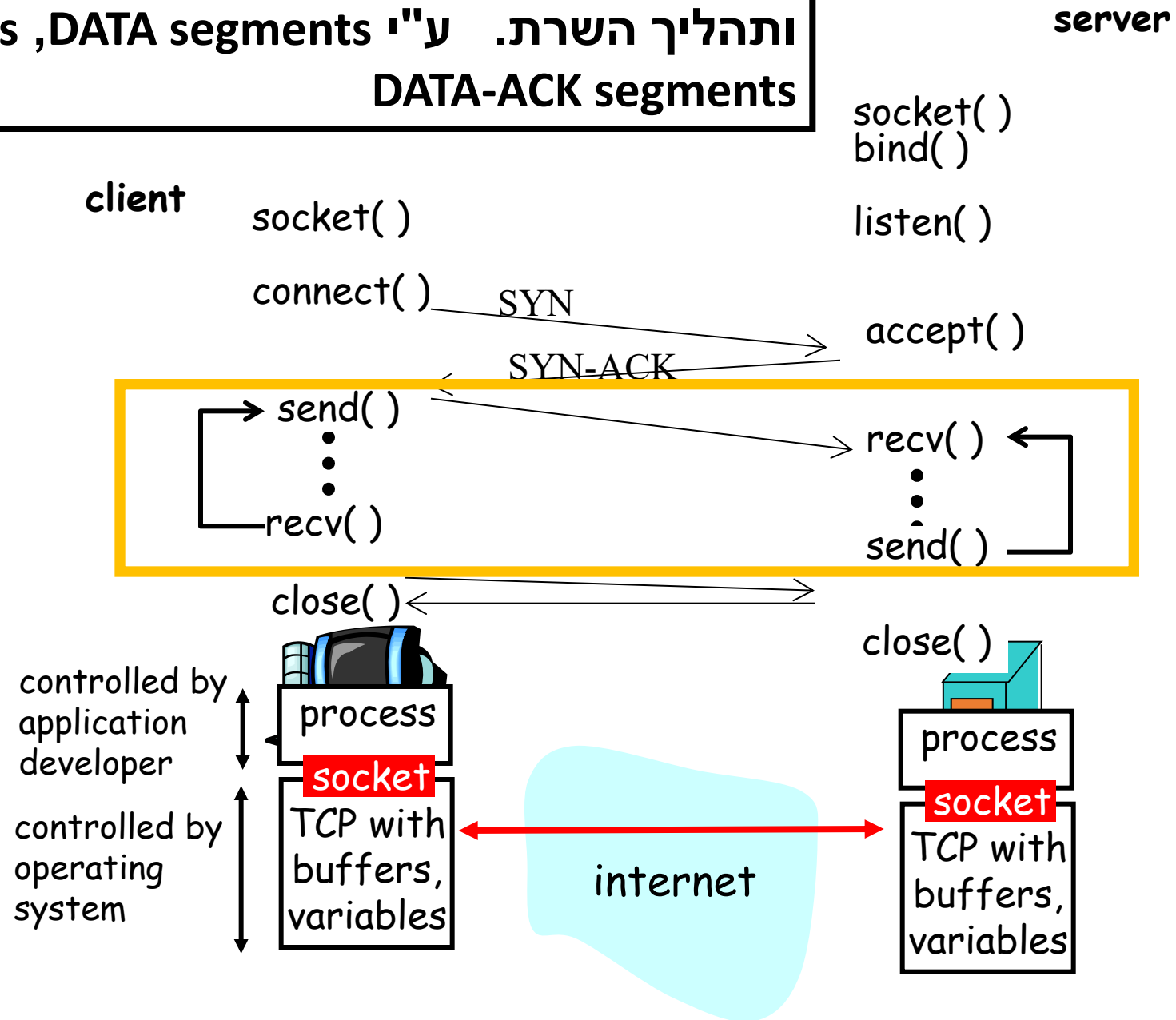
שלב 1 – הקמת קישור (= לחיצת יד)

client

server



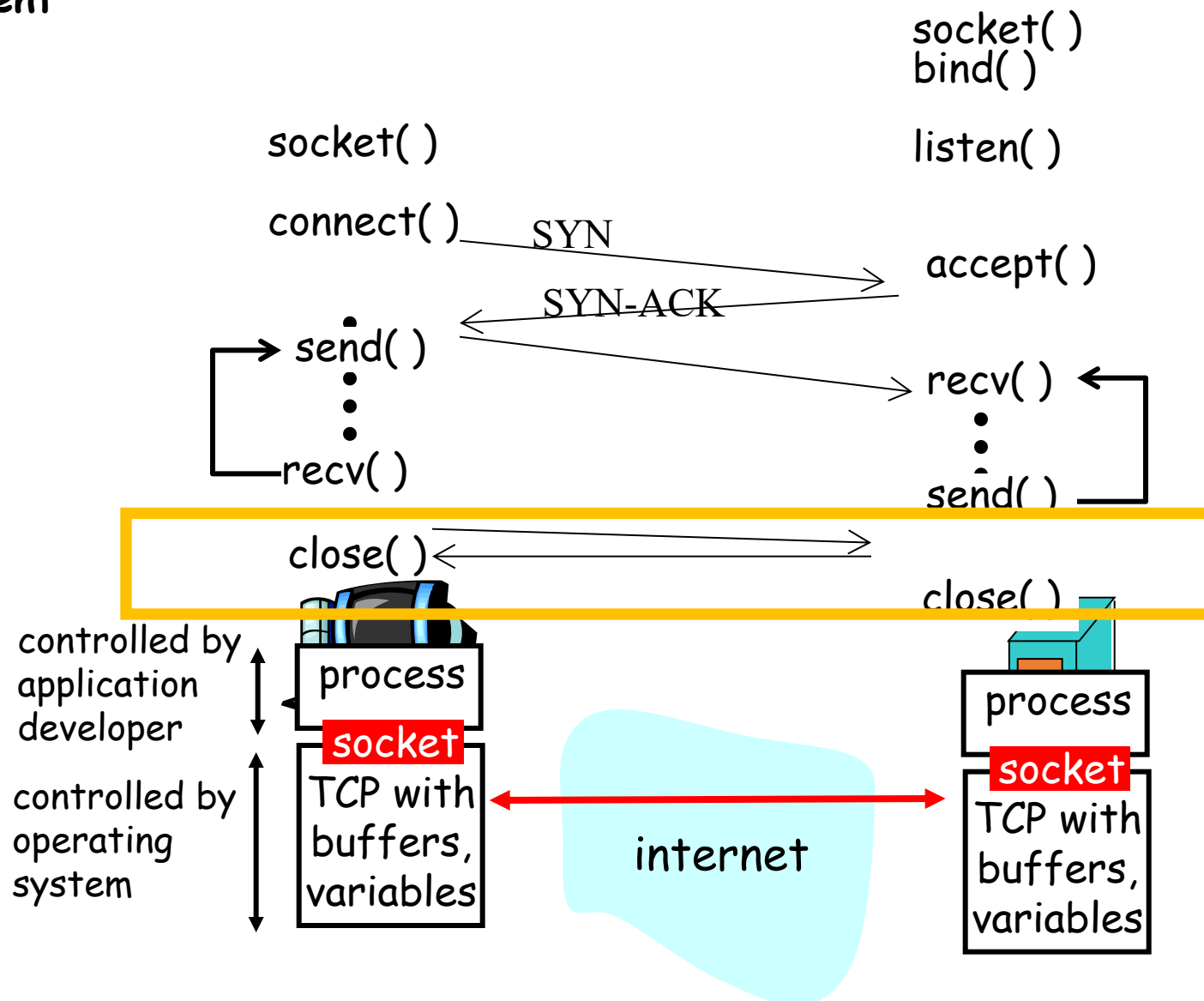
שלב 2 – החלפת הודעות בקשה ותשובה בין תהליך הלקוח ותהליך השרת. ע"י DATA segments ,ACK segments ו- DATA-ACK segments



שלב 3 – סגירת הקישור

client

server

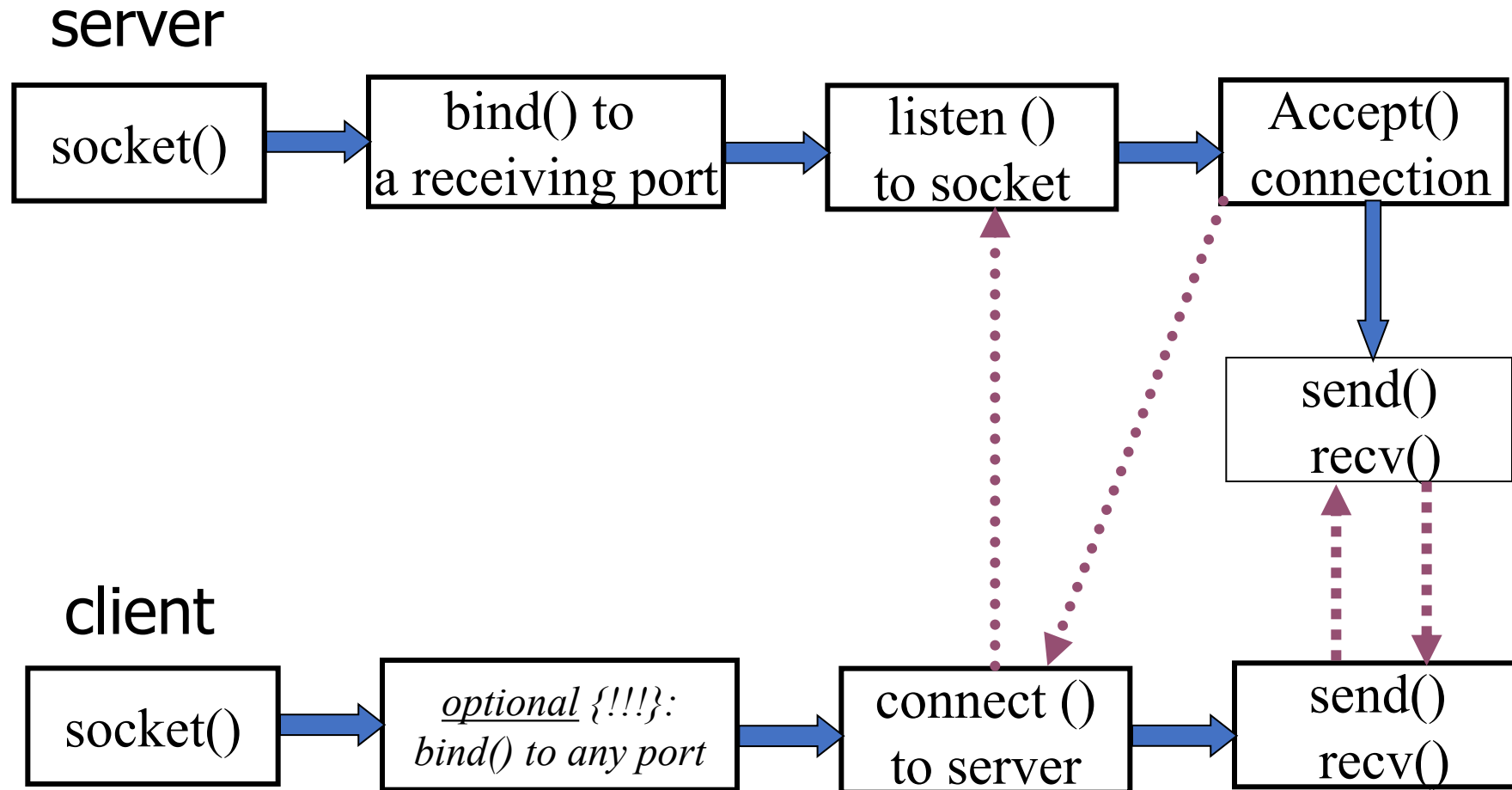


TCP vs. UDP

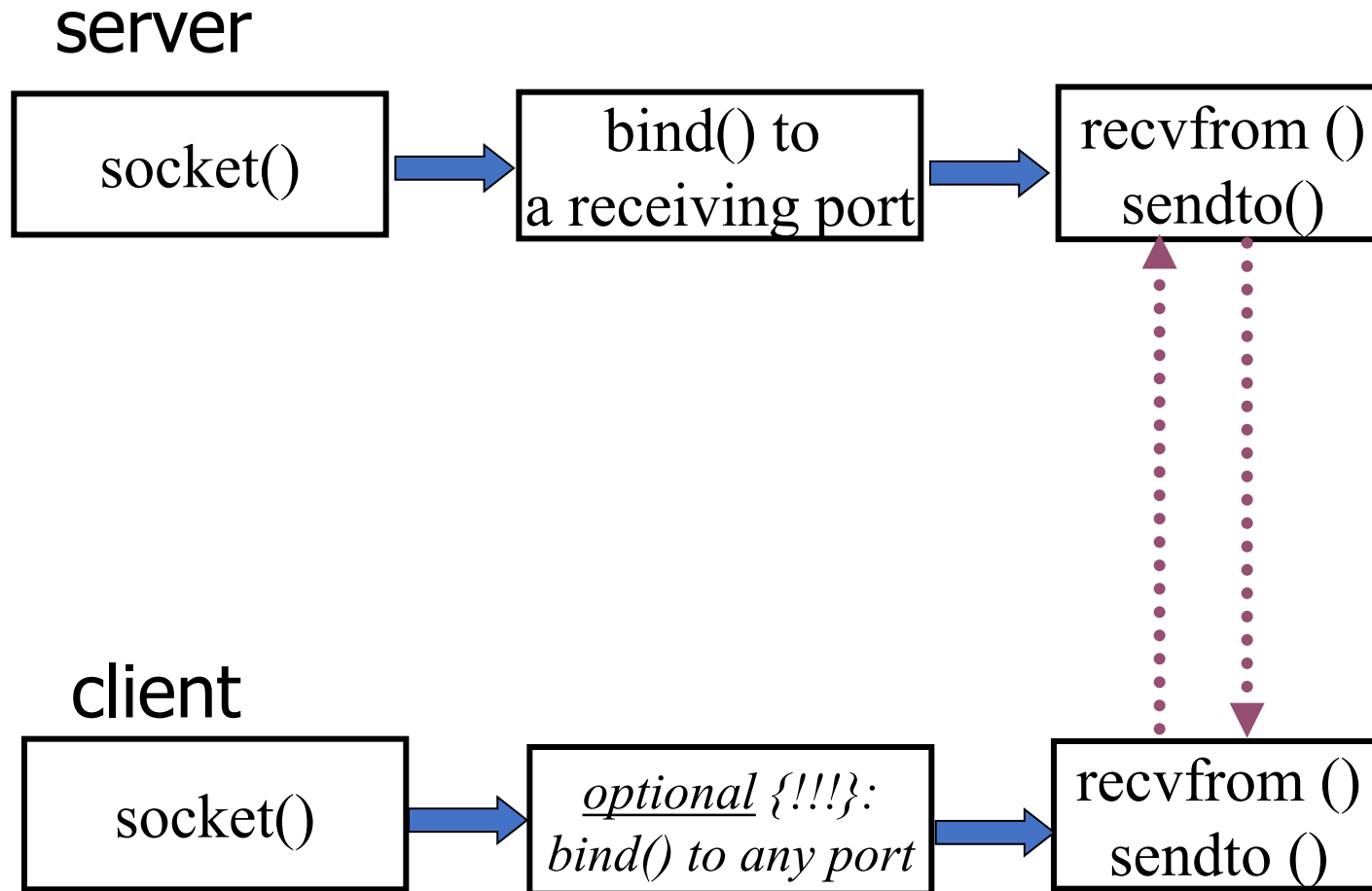
קישורי TCP לעומת קישורי UDP

למרות שהדיון הנוכחי שלנו עוסק ב-TCP, לצורך השוואה ראשונית של TCP ו-UDP נביא להלן:
- את סדרת הצעדים הדרושה להקמת קישור TCP ולהחלפת הודעות 'באמצעותו'
- לעומת סדרת הצעדים הדרושה להקמת קישור UDP ולהחלפת הודעות 'באמצעותו'

TCP - Client-Server communication via a TCP connection

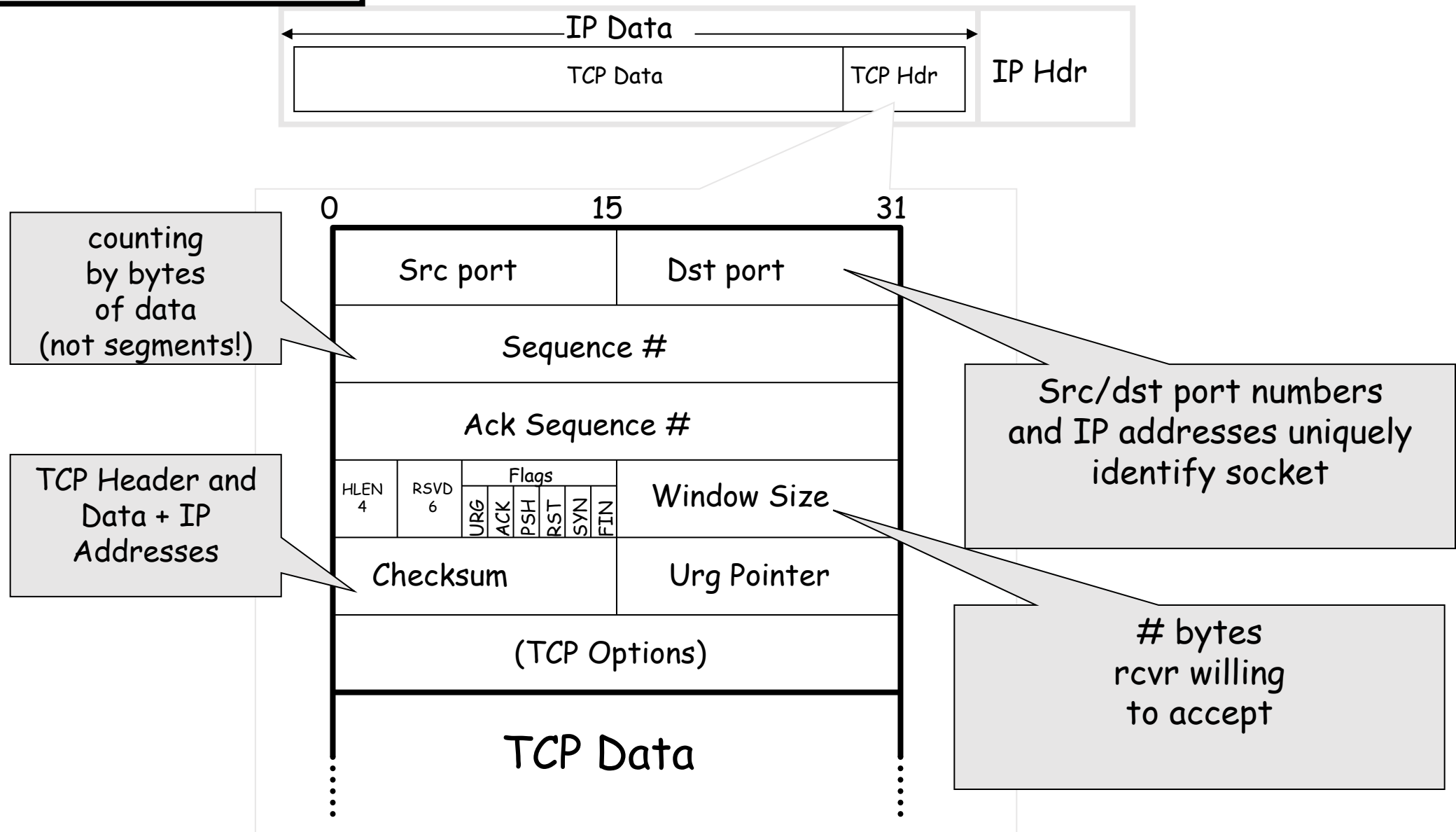


UDP - Client-Server communication via a UDP connection



The TCP Header

The TCP Segment Format



Header Flags



URG – Urgent data

if it's set to 1 it indicates that the urgent pointer is valid

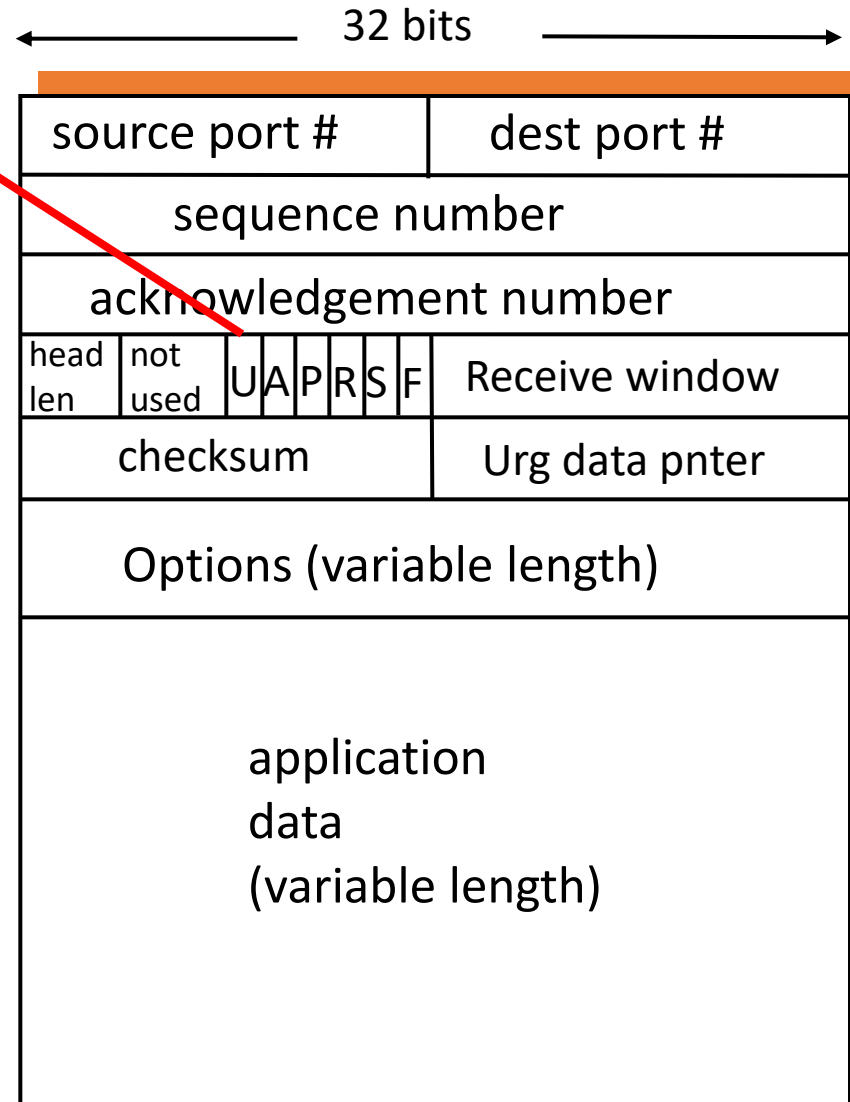
That is: value in the Urgent Pointer field refers to a location in the byte stream (a byte offset from the current sequence number) at which the urgent data can be found.

the following bytes contain an urgent data in the range:

$\text{SeqNo} \leq \text{urgent data} \leq \text{SeqNo} + \text{urgent pointer}$

it tells the receiving side to handle the data pointed by the Urgent Pointer field as quickly as possible.

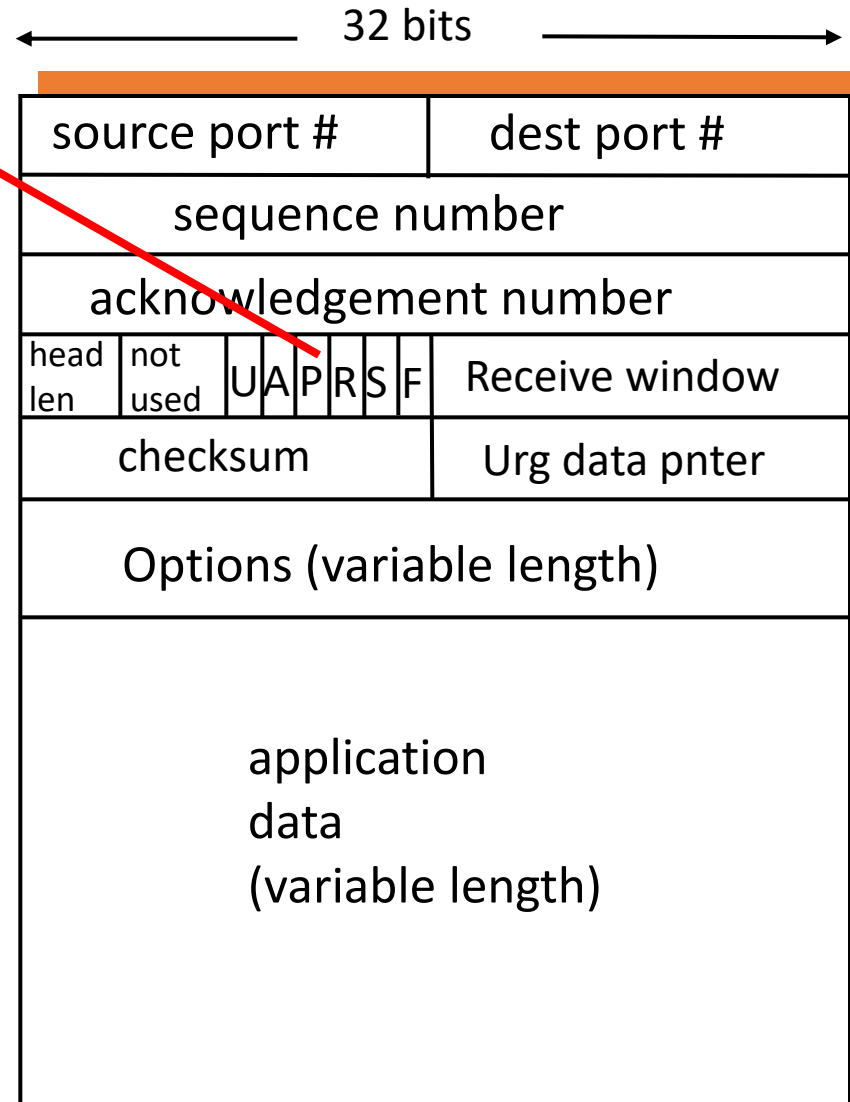
(generally not used)





PSH – Push Flag

Notification from sender to the receiver that the receiver is requested to deliver the data to the application upon arrival and not buffer it.





RST – Reset the connection

The flag causes the receiver to reset the connection

Receiver of a RST terminates the connection and indicates higher layer application about the reset.

Used when the the connection is confused or can't be established.

